SOLAR▸SWARM

Technical Reference
Martin Pham | Perth, AU

System Architecture

3-Tier Agent Hierarchy

```
TIER 1 — THE GENERAL
  master_agent.py
  Runs every 6 hours. Owns strategy.
  ├── Generates experiment ideas via GPT-4o (3 per cycle)
  ├── Scores each idea: market_signal + competitive_gap + execution_speed + revenue_path
  ├── Runs red team analysis (devil_score 1–10)
  ├── Routes: auto_proceed (>8.5) / human_gate (5.0–8.5) / auto_kill (<5.0)
  └── Allocates budget via Kelly Criterion → assigns bucket (exploit/explore/moonshot)

TIER 2 — DEPARTMENT HEADS (run daily at 09:00 UTC)
  research_agent.py   — manages research task queue, spawns prospect/market/competitor tasks
  content_agent.py    — manages content task queue (ad copy, email sequences, SMS, LinkedIn)
  analytics_agent.py  — analyses leads, experiments, budget burn; checks circuit breakers

TIER 3 — WORKERS (stateless, self-terminating)
  worker.py           — picks up next queued task, executes it, posts pheromone signal, exits
  qualification_agent — scores solar leads 1–10 via GPT-4o or rule-based fallback
  proposal_agent      — generates 3-section proposals, saves to proposals/
  solar_research_agent — profiles solar companies for outreach
  report_agent        — weekly client performance reports, saves to reports/
  red_team_agent      — devil's advocate analysis of experiment ideas

SUPPORT AGENTS
  scout_agent.py      — daily 08:00 UTC, discovers new solar company prospects
  mutation_engine.py  — Monday 22:30 UTC, evolves underperforming experiments via GPT-4o
  ab_tester.py        — daily 10:00 UTC, evaluates running A/B tests for winners
```

Capital Allocation Flow

```
confidence_score (0–10)
        |
        ▼
  score_experiment()        ← market_signal, competitive_gap, execution_speed, revenue_path
        |
        ▼
  red_team_analyse()        ← devil_score (1–10)
  adjust_confidence()       ← if devil_score > 6: confidence -= (devil_score - 6) × 0.5
        |
        ▼
  routing decision:
    > 8.5  → auto_proceed
    5.0–8.5 → human_gate  (Slack alert sent, approval required)
    < 5.0  → auto_kill
        |
        ▼
  calculate_budget()        ← 25% fractional Kelly
    p = 0.2 + (confidence/10 × 0.65)    # win probability
    f* = (b×p - q) / b                   # full Kelly (b=3.0 default)
    f_actual = f* × 0.25                 # 25% fractional
    budget = f_actual × WEEKLY_BUDGET_AUD
        |
        ▼
  assign_bucket()
    devil ≥ 7 AND confidence ≥ 7 → moonshot
    confidence ≥ 8 AND devil < 5 → exploit
    otherwise                    → explore
        |
        ▼
  can_allocate()            ← checks bucket remaining vs requested amount
        |
        ▼
  INSERT experiments        ← status: approved / pending
        |
        ▼
  circuit_breaker check     ← after each General cycle
```

Budget Buckets (of WEEKLY_BUDGET_AUD):

BUCKET	ALLOCATION	PURPOSE
exploit	60%	Scaling proven winners
explore	30%	72-hour test experiments
moonshot	10%	High-risk, high-reward ideas

Memory System

Hot Memory — SQLite (`swarm.db`)

Sub-second access to live state. All agents read/write directly.

- `get_active_experiments()` — experiments with status `approved` or `running`
- `get_pending_experiments()` — awaiting human approval
- `get_budget_used_this_week()` — SUM of budget_allocated, last 7 days
- `get_consecutive_failures()` — current failure streak from experiments table
- `enqueue_task()` / `get_next_task()` — task queue operations
- `post_pheromone()` / `get_active_pheromones()` — signal tracking
- `apply_pheromone_decay()` — 50% weight loss per day after 7 days
- `get_circuit_breaker_state()` — most recent unresolved circuit_breaker_log row

Warm Memory — JSON files (`memory/knowledge/`)

Richer structured records for pattern recognition. Slower than SQLite.

FILE	CONTENT
`experiments.json`	All completed experiment outcomes (status, revenue, ROI, learnings)
`learnings.json`	Actionable insights from past experiments, sorted by confidence
`verticals.json`	Market intelligence per vertical (solar_australia, etc.)

Key functions: `save_experiment_outcome()`, `save_learning()`, `get_winning_patterns()`, `get_all_learnings()`, `get_vertical_knowledge()`.

Cold Ledger — Append-only SQLite (`cold_ledger` table)

Every significant decision is written here and **never updated or deleted**. It is the system's audit trail.

Event types written to the ledger:

EVENT TYPE	TRIGGER
`EXPERIMENT_CREATED`	General creates new experiment
`EXPERIMENT_APPROVED`	Human or auto approval
`EXPERIMENT_KILLED`	Auto-kill or rejection
`CIRCUIT_BREAKER_YELLOW/ORANGE/RED`	Threshold breach
`LEAD_QUALIFIED`	Qualification agent scores a lead
`PHEROMONE_SIGNAL`	Worker posts a signal
`RED_TEAM_ANALYSIS`	Red team analysis completed
`RESEARCH_CYCLE`	Research department head run
`ANALYTICS_CYCLE`	Analytics department head run
`VOICE_CALL_COMPLETE`	Post-call processor finishes
`MUTATION_CYCLE`	Mutation engine completes

Message Bus Design

File: `bus/message_bus.py` — backed by the `message_bus` SQLite table.

All inter-agent communication goes through this bus. No direct agent-to-agent calls.

Priority order: `CRITICAL` → `HIGH` → `NORMAL` → `LOW`

Message types:

TYPE	PURPOSE
`TASK`	Request another agent to do work
`REPORT`	Return results of completed work
`ALERT`	Urgent notification requiring attention
`ACK`	Acknowledge receipt of a `requires_ack` message
`KILL`	Instruct agent to stop current work
`QUERY`	Request information
`RESPONSE`	Response to a QUERY

Message lifecycle: `queued` → `processing` → `complete` / `failed` / `expired`

Messages expire after 6 hours if unread (`expire_old_messages()` runs every 6 hours via scheduler).

Key functions:

```
bus.post(from_agent, to_queue, msg_type, payload, priority, ttl_cycles)
bus.receive(queue_name, msg_type=None)       # fetch + mark processing
bus.receive_all(queue_name, limit=20)        # drain queue
bus.complete(msg_id)
bus.fail(msg_id, reason)
bus.ack(original_msg_id, from_agent)
bus.queue_depth(queue_name)                  # counts per status
```

Data Flow Diagram

```
External World
      │
      ├── GHL Webhook (new lead) ──────────────────────────┐
      │                                                     │
      ├── Retell AI (inbound call) ────────────────────┐    │
      │                                                 │    │
      └── Web Scraper / API Poller / Social Signal ────┐    │    │
                                                       │    │    │
                                                       ▼    ▼    ▼
                                              ┌──────────────────┐
                                              │  SQLite (swarm.db) │
                                              │                  │
                                              │ collected_data   │
                                              │ leads            │
                                              │ call_logs        │
                                              │ message_bus      │
                                              │ experiments      │
                                              │ pheromone_signals │
                                              │ kg_entities      │
                                              │ crm_cache        │
                                              └──────────────────┘
                                                       │
                                                       │
                                                       │
```

Scheduler Job Table

All times are UTC. Configured in `main.py` `setup_scheduler()`.

JOB ID	TRIGGER	WHAT IT DOES
`general`	Every 6h	The General's strategic planning cycle: generate ideas, score, route, allocate
`department_heads`	Daily 09:00	Run research_agent, content_agent, analytics_agent in sequence
`retrospective`	Monday 22:00	Weekly retrospective: analyse last 7 days, save learnings to warm memory
`pheromone_decay`	Daily 00:00	Apply 50% decay to pheromone signals older than 7 days
`scout_agent`	Daily 08:00	Discover new solar company prospects from scraped data and social signals
`research_engine`	Daily 06:00	Research orchestrator processes queued research tasks
`data_collection`	Every 4h	Web scraper, API poller, social signal, price monitor collect fresh data
`pipeline_processor`	Every 4h (+30min offset)	Deduplicate, enrich, and route signals from collected_data to message bus
`ab_evaluator`	Daily 10:00	Check running A/B tests; declare winners where sample size is sufficient
`mutation_engine`	Monday 22:30	Evolve underperforming experiments via GPT-4o; kill fatally bad ones
`explore_monitor`	Every 2h	Drive 72-hour explore protocol lifecycle; auto-kill expired experiments
`bus_expiry`	Every 6h	Expire stale messages on the message bus (>6h old queued messages)
`crm_sync`	Every 30min	Pull live CRM data into `crm_cache` table; update board-state.json